

2안 · 4안 발표 대본

기존 캡스톤 아이디어 발표자료(5개 후보) 중 2안(원본 서류 보존연한 관리)과 4안(장애 대응 당번 관리) 두 개 안에 대한 대본만 정리했습니다. 슬라이드 원문의 표현을 그대로 살려서 구성했습니다.

IDEA 02 · 공공·의료·금융

원본 서류의 보존연한 관리 및 파기 이행 추적 시스템

담당자가 바뀌어도 서류의 법정 기한을 놓치지 않게 하는 사내 시스템 · 내가 불편해서 개발

두 번째로 소개해 드릴 아이디어는, 원본 서류의 보존연한을 관리하고 파기 이행을 추적하는 시스템입니다. 담당자가 바뀌어도 서류의 법정 기한을 놓치지 않게 하는 사내 시스템이며, 제가 직접 겪은 불편함에서 출발한 아이디어입니다.

2안 · SLIDE 01 · ①아이디어 요지

법정 보존연한은 담당자의 재직 기간보다 긴 경우가 많습니다. 그래서 서류를 만든 사람과 파기할 사람이 다르고, 인수인계 과정에서 기한 정보가 유실됩니다.

저희가 제안하는 시스템은 문서 내용은 저장하지 않고, 관리번호·생성일·보존연한만 등록받아 기한을 자동 산출합니다. 담당자가 교체되어도 이행 책임이 끊기지 않도록 관리합니다.

내용을 취급하지 않기 때문에 기밀을 다루는 조직도 도입 부담이 낮고, **내부망 설치형**으로 제공되어 데이터가 외부로 나가지 않습니다.

↓ 다음 슬라이드로

2안 · SLIDE 02 · ②배경 및 구현 가능성

이 아이디어는 군 복무 중 보안 문서를 취급하면서 직접 겪은 경험에서 출발했습니다. 서면 대장만으로는 문서의 생성부터 파기까지 추적이 어렵다는 것을 겪었고, 보관 기한이 수년 단위이다 보니 생성자와 파기자가 다른 경우가 대부분이었습니다.

이 문제의 핵심은 문서의 내용이 아니라 **기한**이라고 판단했습니다. 조사해보니 의료·상업·세무·공공 분야 모두 같은 구조의 문제를 가지고 있었고, 다만 보존 기한의 기산점을 정하는 방식이 분야마다 달랐습니다.

기존 사례와의 차이를 말씀드리면, 개인정보 자동화 솔루션은 전자 파일을 탐지하고 삭제하는 방식으로 동작합니다. 그런데 전자문서법상 예외로 분류되는 서명 계약서, 등기, 소송 서류 같은 물리 원본은 여전히 존재하고, 그 기한 관리는 사각지대로 남아 있습니다.

↓ 다음 슬라이드로

2안 · SLIDE 03 · ②시스템 구조 및 동작 흐름

화면의 구조도를 보시면, 담당자·상급자·감사 담당이 접근 제어를 거쳐 시스템에 진입하고, 핵심 구현인 **기한 산출 엔진**이 조직별 규칙을 바탕으로 파기 예정일을 계산합니다. 이후 알림 스케줄러가 사전 통지와 미확인 시 승격, 다중 경로 알림을 처리하고, 마지막으로 기록 저장소에 관리번호·날짜·담당자·이행 이력이 남습니다. 문서 내용 자체는 저장하지 않습니다.

동작 흐름은 문서 등록 이후 규칙 조회를 거쳐 기산점을 계산하고, 파기 예정일을 산출합니다. 기한이 임박하고 보류 사유가 없으면 담당자에게 통지하고, 기간 내 응답이 없으면 상급자에게 승격한 뒤 재통지합니다. 응답이 오면 파기를 승인하고, 이행 기록과 감사 증빙을 보관합니다.

↓ 다음 슬라이드로

2안 · SLIDE 04 · ③아이디어 상용화 가능성

대상 고객	중소 의료기관, 금융·법무·회계 사무소, 공공기관, 제조·건설 안전관리 부서입니다.
현재 방식	엑셀 파일, 종이 대장, 담당자의 기억에 의존하고 있습니다.
지불 근거	개인정보보호법 위반 시 3천만원 이하 과태료가 부과될 수 있고, 보존에 실패하면 분쟁 시 증빙이 불가능해 집니다.
제공 형태	내부망 설치형이며, 라이선스와 연 유지보수 형태로 제공합니다.

기술적으로 풀어야 할 과제도 있습니다. 기한 산출 규칙 엔진은 법령마다 기산점이 달라서(생성일, 종결일, 회계연도 절상 등) 조직별로 규칙을 정의할 수 있는 구조가 필요합니다. 또한 알림의 신뢰성과 권한 승계도 중요한데, 미확인 시 자동 승격되고 상급자가 수직으로 조회할 수 있어야 하며, 전임에서 후임으로 이어지는 시간축 승계도 구현해야 합니다.

↓ 다음 슬라이드로

2안 · SLIDE 05 · ④개발환경 및 도구

역할 기반 접근 제어	ASP.NET Core Identity 또는 Spring Security
규칙 엔진 설계	직접 구현
스케줄링·재시도	Quartz.NET 또는 Hangfire
알림 채널 연동	SMTP, 사내 메신저 연동 API
감사 로그·무결성	해시 체이닝, 추가 전용 테이블
데이터베이스·이력 관리	PostgreSQL

↓ 다음 슬라이드로

- **2026.09** — 분야별 보존연한 법령 조사, 규칙 엔진 설계, 단일 문서 등록·기한 산출 프로토타입
- **2026.10** — 프로토타입 완성, 기한 산출 정확성 검증
- **2026.11** — 알림·승격 엔진 본구현
- **2026.12** — 인수인계 절차, 권한 체계 구현
- **2027.01~02** — 감사 대응 기록, 실사용 시나리오 검증
- **2027.03** — 최종 정리, 발표

여기까지가 2안, 원본 서류 보존연한 관리 시스템에 대한 설명이었습니다. 이어서 4안을 소개해 드리겠습니다.

IDEA 04 · 서비스 운영

소규모 운영팀을 위한 장애 대응 당번 관리 시스템

새벽에 서버가 멈췄을 때 누군가는 반드시 받도록 만드는 국내 환경 기반 온콜 도구 · 기 개발된 것이 있으나 불편해서 개선

네 번째로 소개해 드릴 아이디어는, 소규모 운영팀을 위한 장애 대응 당번 관리 시스템입니다. 새벽에 서버가 멈췄을 때 누군가는 반드시 응답하도록 만드는, 국내 환경에 맞춘 온콜 도구입니다. 이미 나와 있는 도구가 있지만, 저희가 직접 써 보며 느낀 불편함을 개선하고자 기획했습니다.

업무 시간 외 장애 알림을 팀 전체에 보내면 책임이 분산되어 아무도 받지 않는 문제가 있습니다. 소규모 팀은 전용 도구가 비싸서 단체 대화방에 의존하는 경우가 많습니다.

저희 시스템은 당번을 자동으로 순환시키고, 응답이 없으면 정해진 시간 뒤 다음 담당자로 승격시키며, 메신저에서 전화까지 수단을 단계적으로 높입니다.

국내 업무용 메신저와 문자·음성 발신을 조합해 인당 과금이 아닌 **정액 구조**로 제공하고, 응답 이력을 기록해 사후 분석이 가능하게 합니다.

↓ 다음 슬라이드로

4안 · SLIDE 02 · ②배경 및 구현 가능성

24시간 서비스는 업무 시간 외 장애를 누가 받을지가 항상 문제입니다. 전체 알림은 책임 분산 현상을 만들어 결국 아무도 받지 않게 됩니다.

대규모 조직은 전용 도구를 쓰지만, 대부분 인당 과금 구조라 팀이 커질수록 비용이 급증하고, 국외 메신저를 전제로 하기 때문에 국내 팀과는 맞지 않습니다.

기존 사례와의 차이를 조사해보니, PagerDuty나 Opsgenie 등 열 곳 이상을 확인했는데 전부 Slack 같은 국외 메신저를 전제로 하고 있었습니다. 국내 알림톡은 간헐적 발송이라 부적합하고, 개인 메신저 봇은 공식 API가 없습니다. 그래서 저희는 업무용 메신저와 국내 번호 문자·음성을 조합해 이 공백을 메우고자 합니다.

↓ 다음 슬라이드로

4안 · SLIDE 03 · ②시스템 구조 및 동작 흐름

구조도를 보시면, 기존에 사용 중인 모니터링 도구가 웹hook으로 알림을 보내면, 알림 수신 단계에서 중복을 제거하고 유사 알림을 그룹화합니다. 이어서 당번 판정 단계가 순환 일정과 부재 여부를 반영해 당번을 정하고, 핵심 구현인 **승격 엔진**이 단계별 대기 후 수단을 상향시킵니다. 1단계 메신저, 2단계 문자, 3단계 전화, 4단계 차순위 담당자 순입니다. 발송 이후에는 응답 기록과 통계로 수신·응답 시각과 처리 단계가 남습니다.

동작 흐름으로 보면, 장애 알림을 수신하면 당번에게 1단계 알림을 보냅니다. 응답이 없고 마지막 수단이 아니라면 다음 수단으로 승격해 재알림하고, 마지막 수단까지 응답이 없으면 차순위 담당자에게 전화합니다. 응답이 오면 기록하고, 처리 단계는 통계에 반영됩니다.

↓ 다음 슬라이드로

4안 · SLIDE 04 · ③아이디어 상용화 가능성

대상 고객	24시간 운영하는 소규모 개발·운영팀입니다. 스타트업, 중소 SI, 운영 대행사가 해당됩니다.
--------------	--

현재 방식	단체 대화방에서 구두로 합의하거나, 고가의 국외 서비스를 사용하고 있습니다.
--------------	--

지불 근거	대응이 지연되면 곧 서비스 중단으로 이어지고, 계약상 가용률을 보장해야 하면 배상 책임까지 발생합니다. 심야 대기는 이직 요인이 되기도 합니다.
--------------	--

제공 형태	클라우드 구독형(정액제)을 기본으로 하고, 보안이 중요한 조직을 위한 설치형도 고려하고 있습니다.
--------------	--

구현 시 특히 신경 써야 할 부분은 두 가지입니다. 첫째, 전달 보장입니다. 알림이 전달되지 않으면 무의미하기 때문에, 발송 실패를 감지하고 재시도해야 하며 시스템 자체의 상태도 외부에서 확인할 수 있어야 합니다. 둘째, 정확한 시각 기반 승격입니다. 서버가 재시작되어도 승격 정보가 소실되거나 중복 발송되면 안 되므로, 지연 작업의 연속성이 필요합니다.

↓ 다음 슬라이드로

지연 작업 스케줄링	Quartz.NET 또는 Hangfire, Redis
웹훅 수신·중복 그룹화	ASP.NET Core, 해시 기반 판정
메시지 발송 연동	카카오워크·네이버웍스 봇 API, 문자·음성 발신 API
상태 기계 설계	직접 구현
당번 순환 알고리즘	직접 구현
자체 감시·가용성	외부 헬스체크, Prometheus

↓ 다음 슬라이드로

- 2026.09 — 카카오워크 등 봇 API·문자·음성 발신 연동 검증, 단일 채널 프로토타입
- 2026.10 — 프로토타입 완성, 전달 성공률 검증
- 2026.11 — 웹훅 수신, 당번 판정, 승격 엔진 본구현
- 2026.12 — 다중 채널 발송, 알림 폭주 대응
- 2027.01~02 — 실사용 시나리오 검증, 보완
- 2027.03 — 최종 정리, 발표

이상으로 2안과 4안, 두 가지 아이디어에 대한 설명을 마치겠습니다. 감사합니다.